Assignment 03:
Pipeline Processor

IC Design and Verification
SSCASEIT -Department of Computer Science

Course:
Computer Architecture

Submitted By:
Sara waheed

Registration No:
Case-035

Objective:

Design, implement and verify a five-stage pipelined RISC-V (RV32I) processor in Verilog incorporating instruction pipelining, hazard detection, data forwarding, branch handling and memory access. Validate its functionality through simulation and waveform analysis.

Design:

1. Five-Stage Pipeline

The processor was implemented using the standard five-stage RISC-V pipeline:

- Instruction Fetch (IF): Fetches instructions from instruction memory using the Program Counter (PC).
- Instruction Decode (ID): Decodes the instruction, generates control signals, reads source registers, and generates the immediate value.
- Execute (EX): Performs ALU operations, branch target calculation, and operand forwarding.
- Memory (MEM): Performs load and store operations using the data memory.
- Write Back (WB): Writes the ALU result, memory data, or PC+4 back to the register file.

Pipeline registers were inserted between every stage (IF/ID, ID/EX, EX/MEM, MEM/WB) to allow multiple instructions to execute simultaneously

2. Modular Design

The processor was divided into independent modules such as:

- Program Counter
- Instruction Fetch
- Instruction Decode
- Register File
- Immediate Generator
- Control Unit
- ALU Control
- ALU
- Execute Stage
- Memory Stage

- Write Back Stage
- Hazard Detection Unit
- Forwarding Unit

3. Pipeline Registers

Pipeline registers were used to store both data and control signals between stages. This ensures that each instruction carries its required information through the pipeline while allowing new instructions to enter every clock cycle.

4. Forwarding Unit

A forwarding unit was implemented to resolve most data hazards without stalling the pipeline. The forwarding multiplexers select operand values from either:

- Register File
- EX/MEM pipeline register
- MEM/WB pipeline register

This reduces unnecessary pipeline stalls and improves performance.

5. Hazard Detection Unit

A hazard detection unit detects load use hazards. When a load instruction is immediately followed by an instruction that requires the loaded value the unit:

- Stops the Program Counter
- Stops the IF/ID pipeline register
- Flushes the ID/EX pipeline register by inserting a bubble

This ensures correct execution while minimizing stalls.

6. Branch Handling

Branch decisions are made in the Execute stage after comparing ALU operands. When a branch is taken, the pipeline is flushed to remove incorrectly fetched instructions, and the Program Counter is updated with the branch target address.

Hazards Encountered

1. Data Hazards

Data hazards occurred when an instruction depended on the result of a previous instruction before it had reached the write-back stage Without forwarding, the second instruction would read the old value of x3.

Solution:

A forwarding unit forwards the latest result from the EX/MEM or MEM/WB pipeline registers directly to the ALU inputs.

2. Load-Use Hazard

A load instruction produces its result only after the Memory stage.

Solution:

The hazard detection unit inserts one stall cycle by freezing the PC and IF/ID register and flushing the ID/EX register.

3. Control Hazards

Control hazards occur because the processor does not know whether a branch is taken until the Execute stage.

Solution:

If a branch is taken, the pipeline flushes incorrect instructions and updates the PC with the branch target

Forwarding Logic:

In a pipelined processor data hazards occur when an instruction requires the result of a previous instruction before it has been written back to the register file. To reduce pipeline stalls and improve performance a forwarding unit is implemented.

The forwarding unit continuously compares the source registers (rs1 and rs2) of the instruction in the Execute (EX) stage with the destination registers (rd) of instructions in the EX/MEM and MEM/WB pipeline registers. If a match is detected and the previous instruction writes to a register the most recent result is forwarded directly to the ALU inputs instead of waiting for the register write-back stage.

The forwarding logic operates as follows:

- No Forwarding (Forward = 00): Operands are taken directly from the register file.
- Forward from MEM/WB (Forward = 01): The operand is forwarded from the write-back stage.
- Forward from EX/MEM (Forward = 10): The operand is forwarded from the EX/MEM pipeline register. This has the highest priority because it contains the most recent result.

This mechanism removes most Read After Write (RAW) data hazards without introducing pipeline stalls.

Forwarding Decision Table:

Condition	ForwardA/ForwardB
No dependency	00
Data available in MEM/WB stage	01
Data available in EX/MEM stage	10

Hazard Detection Logic:

Although forwarding resolves most data hazards it cannot resolve load use hazards. A load instruction (lw) retrieves data from memory during the Memory stage so the required data is not available when the next instruction reaches the Execute stage.

The hazard detection unit monitors the instruction in the ID/EX pipeline register and checks whether it is a load instruction (MemRead = 1). It then compares its destination register (rd_EX) with the source registers (rs1_ID and rs2_ID) of the instruction currently in the Decode stage.

The hazard detection condition is:

If (MemRead_EX = 1) AND

(rd_EX == rs1_ID OR rd_EX == rs2_ID)

then a load use hazard exists. To resolve this hazard, the processor performs the following actions:

- PCWrite = 0: Freezes the Program Counter so that no new instruction is fetched.
- IF_ID_Write = 0: Prevents the IF/ID pipeline register from updating keeping the current instruction in the Decode stage.
- Flush = 1: Clears the ID/EX pipeline register inserting a NOP (bubble) into the Execute stage.

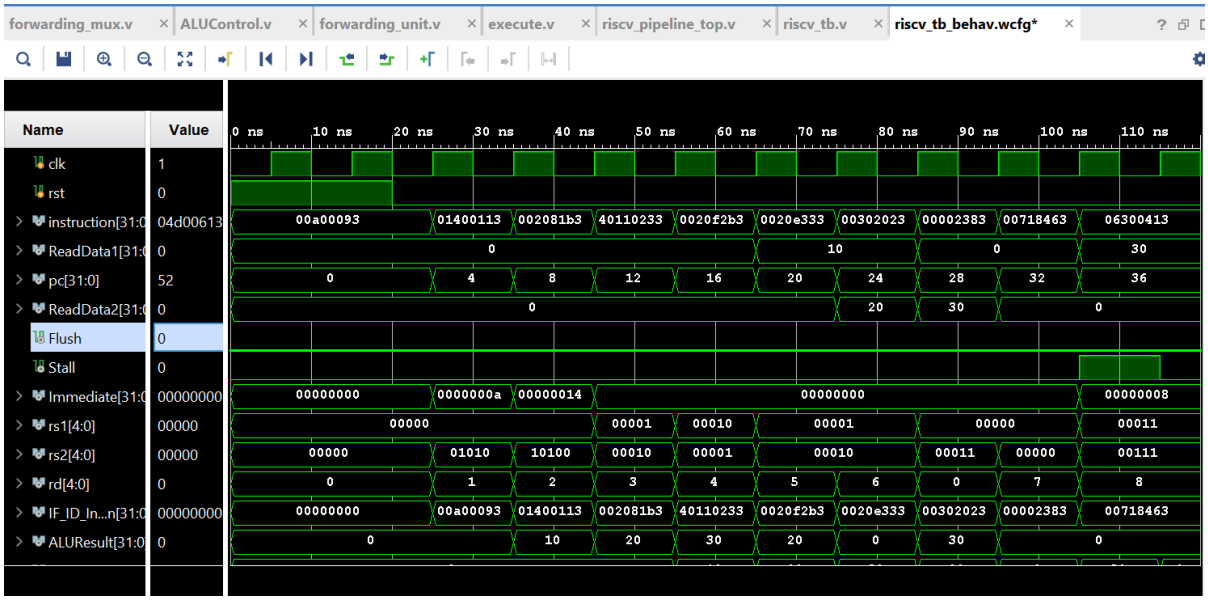
The bubble provides one additional clock cycle for the load instruction to complete the memory access. After the data becomes available the dependent instruction proceeds normally with the correct operand.

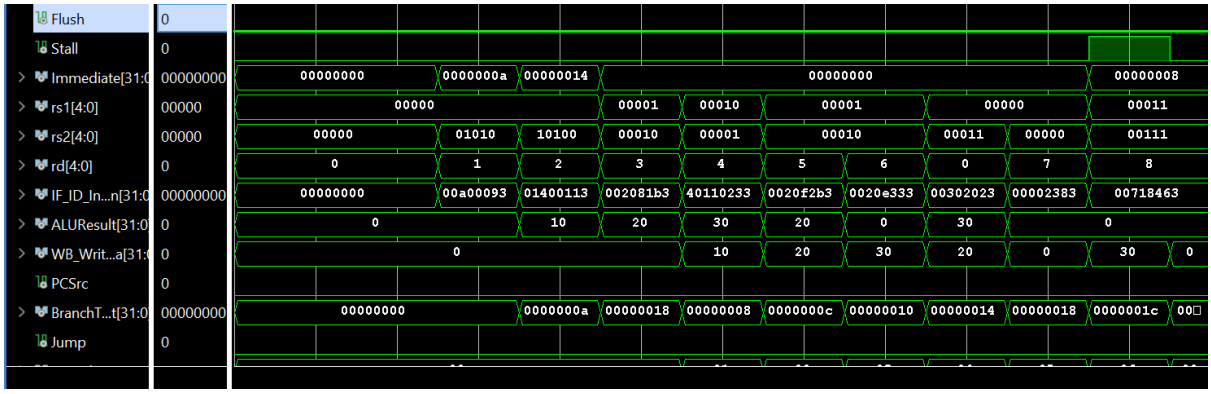
Simulation:

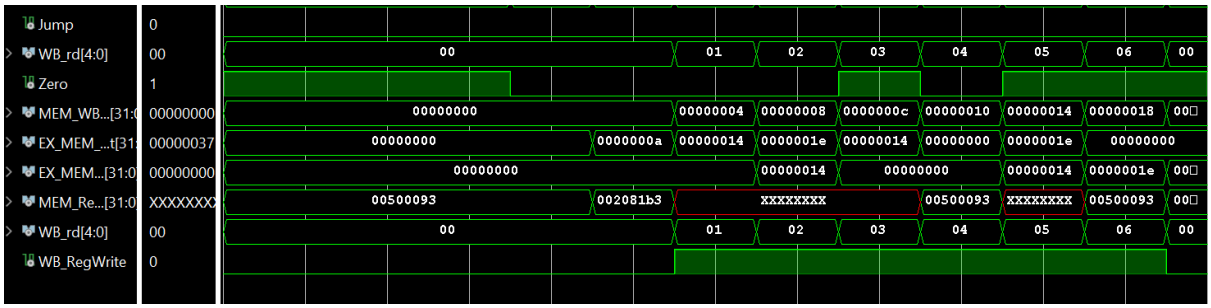
```
# run 1000ns
```

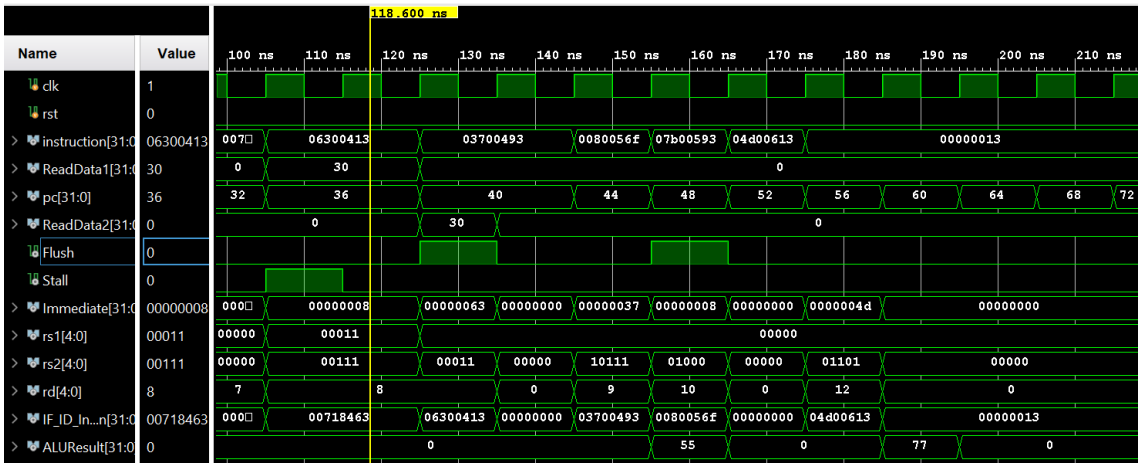
Time	PC	Instr	EX_PC	RD1	RD2	IMM	ALUSrc	ALUOp	ALUCtrl	ALUResult	WB_RegWrite	WB_rd	WBData
25000	00000000	00a00093	00000000	00000000	00000000	00000000	0	00	0000	00000000	0	0	00000000
35000	00000004	01400113	00000000	00000000	00000000	00000000	0	00	0000	00000000	0	0	00000000
45000	00000008	002081b3	00000000	00000000	00000000	0000000a	1	11	0000	0000000a	0	0	00000000
55000	0000000c	40110233	00000004	00000000	00000000	00000014	1	11	0000	00000014	0	0	00000000
65000	00000010	0020f2b3	00000008	00000000	00000000	00000000	0	10	0000	0000001e	1	1	0000000a
75000	00000014	0020e333	0000000c	00000000	00000000	00000000	0	10	0001	00000014	1	2	00000014
85000	00000018	00302023	00000010	0000000a	00000000	00000000	0	10	0010	00000000	1	3	0000001e
95000	0000001c	00002383	00000014	0000000a	00000014	00000000	0	10	0011	0000001e	1	4	00000014
105000	00000020	00718463	00000018	00000000	0000001e	00000000	1	00	0000	00000000	1	5	00000000
115000	00000024	06300413	0000001c	00000000	00000000	00000000	1	00	0000	00000000	1	6	0000001e
125000	00000024	06300413	00000000	00000000	00000000	00000000	0	00	0000	00000000	0	0	00000000
135000	00000028	03700493	00000020	0000001e	00000000	00000008	0	01	0001	00000000	1	7	0000001e
145000	00000028	03700493	00000000	00000000	00000000	00000000	0	00	0000	00000000	0	0	00000000
155000	0000002c	0080056f	00000000	00000000	00000000	00000000	0	00	0000	00000000	0	8	00000000
165000	00000030	07b00593	00000028	00000000	00000000	00000037	1	11	0000	00000037	0	0	00000000
175000	00000034	04d00613	00000000	00000000	00000000	00000000	0	00	0000	00000000	0	0	00000000
185000	00000038	00000013	00000000	00000000	00000000	00000000	0	00	0000	00000000	1	9	00000037
195000	0000003c	00000013	00000034	00000000	00000000	0000004d	1	11	0000	0000004d	0	0	00000000
205000	00000040	00000013	00000038	00000000	00000000	00000000	1	11	0000	00000000	0	0	00000000
215000	00000044	00000013	0000003c	00000000	00000000	00000000	1	11	0000	00000000	1	12	0000004d

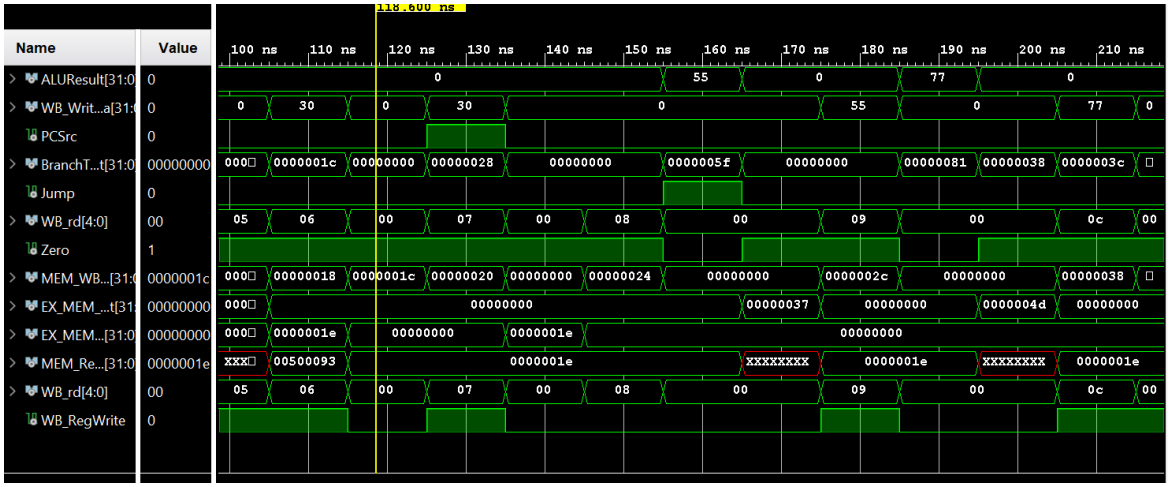
215000	00000044	00000013	0000003c	00000000	00000000	00000000	1	11	0000	00000000	1	12	0000004d
225000	00000048	00000013	00000040	00000000	00000000	00000000	1	11	0000	00000000	1	0	00000000
235000	0000004c	00000013	00000044	00000000	00000000	00000000	1	11	0000	00000000	1	0	00000000
245000	00000050	00000013	00000048	00000000	00000000	00000000	1	11	0000	00000000	1	0	00000000
255000	00000054	00000013	0000004c	00000000	00000000	00000000	1	11	0000	00000000	1	0	00000000
265000	00000058	xxxxxxxx	00000050	00000000	00000000	00000000	1	11	0000	00000000	1	0	00000000
275000	0000005c	xxxxxxxx	00000054	00000000	00000000	00000000	1	11	0000	00000000	1	0	00000000
285000	00000060	xxxxxxxx	00000058	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	1	0	00000000
295000	00000064	xxxxxxxx	0000005c	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	1	0	00000000
305000	00000068	xxxxxxxx	00000060	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx
315000	0000006c	xxxxxxxx	00000064	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx
325000	00000070	xxxxxxxx	00000068	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx
335000	00000074	xxxxxxxx	0000006c	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx
345000	00000078	xxxxxxxx	00000070	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx
355000	0000007c	xxxxxxxx	00000074	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx
365000	00000080	xxxxxxxx	00000078	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx
375000	00000084	xxxxxxxx	0000007c	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx
385000	00000088	xxxxxxxx	00000080	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx
395000	0000008c	xxxxxxxx	00000084	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx
405000	00000090	xxxxxxxx	00000088	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx
415000	00000094	xxxxxxxx	0000008c	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx
425000	00000098	xxxxxxxx	00000090	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx
435000	0000009c	xxxxxxxx	00000094	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx
445000	000000a0	xxxxxxxx	00000098	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx
455000	000000a4	xxxxxxxx	0000009c	xxxxxxxx	xxxxxxxx	00000000	0	00	0000	xxxxxxxx	0	x	xxxxxxxx


forwarding_mux.v
ALUControl.v
forwarding_unit.v
execute.v
riscv_pipeline_top.v
riscv_tb.v
riscv_tb_behav.wcfg*
Name
Value
0 ns
10 ns
20 ns
30 ns
40 ns
50 ns
60 ns
70 ns
80 ns
90 ns
100 ns
110 ns
clk
rst
instruction[31:0]
ReadData1[31:0]
pc[31:0]
ReadData2[31:0]
Flush
Stall
Immediate[31:0]
rs1[4:0]
rs2[4:0]
rd[4:0]
IF_ID_In...n[31:0]
ALUResult[31:0]

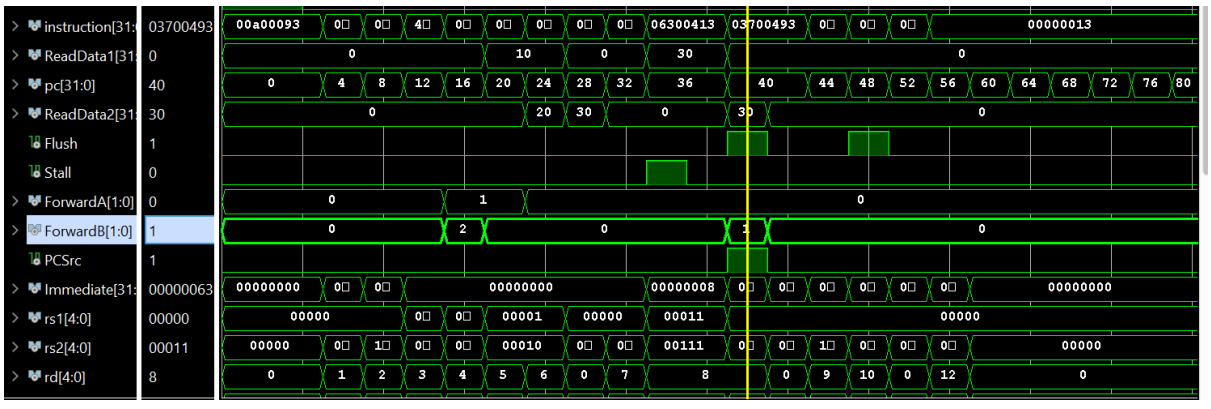

Flush
Stall
Immediate[31:0]
rs1[4:0]
rs2[4:0]
rd[4:0]
IF_ID_In...n[31:0]
ALUResult[31:0]
WB_Writ...a[31:0]
PCSrc
BranchT...t[31:0]
Jump


Jump
WB_rd[4:0]
Zero
MEM_WB...[31:0]
EX_MEM_...t[31:0]
EX_MEM...[31:0]
MEM_Re...[31:0]
WB_rd[4:0]
WB_RegWrite





Hazards and solution:



Conclusion:

The implemented processor successfully executes a subset of the RV32I instruction set using a five stage pipelined architecture. Pipeline registers, forwarding logic, hazard detection and branch flushing were implemented to improve performance while maintaining correct instruction execution. Functional verification was performed using a Verilog testbench and waveform analysis in Vivado.